Lecture 2 — Polynomial Techniques

> World Finals Level · Prerequisites: *nationals-07* (FFT/NTT, matrix exponentiation, linear recurrences), *week-11* (modular inverses), *week-30* (combinatorics) · Estimated time: 2 × 90 minutes

1. Overview

In *nationals-07* you learned to multiply two polynomials in O(n log n) with FFT/NTT. At World Finals and Codeforces Div1 E/F level, multiplication is the bottom layer of a whole algebra on sequences:

- **Formal power series operations**: inverse, logarithm, exponential and power, all in O(n log n) using Newton's iteration.
- **Combinatorial meaning**: with exponential generating functions, log turns "all structures" into "connected structures" and exp goes the other way.
- **Berlekamp–Massey (BM)**: recover the shortest linear recurrence from the first terms of a sequence, then compute the 10^{18}-th term.
- **Lagrange interpolation**: when an answer is a polynomial in `n` of degree `d`, compute `d + 1` values by brute force and evaluate at any `n` in O(d).

Everything is done modulo the NTT prime $p = 998244353 = 119 \cdot 2^{23} + 1$ (primitive root 3) unless stated otherwise. When a problem uses 10^9+7, either the technique needs no NTT (BM, Lagrange), or you need arbitrary-modulus convolution (Section 3.3).

2. NTT, briefly

A length-n NTT evaluates a polynomial at the n powers of an n-th root of unity in F_p. That root exists when n divides $p - 1 = 119 \cdot 2^{23}$, so the transform length can be any power of two up to 2^{23}. The reference implementation (Section 8) uses the iterative bit-reversal form with a precomputed table of roots. The inverse transform is the forward transform followed by reversing `a[1..n-1]` and dividing by `n`. Two implementation details matter for speed:

- Precompute the roots once (`rt[k + j] = w_{2k}^j`) instead of updating `w *= wlen` inside the innermost loop, which costs a modular multiplication and creates a dependency chain.
- For tiny inputs (min size ≤ 32), schoolbook multiplication is faster than three transforms. This matters because the Newton loops below start with sizes 1, 2, 4, ….

3. NTT applications

3.1 Direct convolutions

- **Sum counting.** For two multisets A and B with values up to V, the number of pairs with `a + b = s`, for every `s`, is the convolution of their frequency arrays.
- **Pattern matching with wildcards.** Encode wildcards as 0. Pattern position i matches text position j + i when `p_i t_{j+i} (p_i − t_{j+i})² = 0`. Summed over i, this expands into three convolutions (of p^3 with t, p^2 with t^2, and p with t^3) after reversing the pattern. Codeforces 528D *Fuzzy Search* is a variant that needs one convolution per letter.
- **Divisor-type sums.** In WF 2015 *Tile Cutting*, the number of tilings of each area is a convolution of the divisor-count function with itself, so one big multiplication answers every query.

3.2 Products of many polynomials

To compute `Π_i (1 + a_i x)`, or any product of many small polynomials, use divide and conquer: split the factors into halves, recurse, and multiply the two results. Every level of the recursion multiplies polynomials whose total degree is n, so it costs O(n log n), and there are log n levels: **O(n log² n)**. A heap that always multiplies the two smallest polynomials achieves the same bound when the factors have very different degrees.

3.3 Arbitrary modulus

For a modulus like 10^9+7, the true coefficients of a product can reach $n\cdot(10^9)^2 \approx 10^{23}$, which does not fit in any single NTT prime. There are two standard fixes:

1. Convolve modulo three NTT primes (for example 998244353, 167772161 and 469762049) and recombine each coefficient with the Chinese Remainder Theorem. This takes nine NTTs and needs care with 128-bit intermediates.
2. Split each coefficient as $a = a_1\cdot 2^{15} + a_0$ and use complex FFT with `long double` or careful `double` rounding (KACTL's `convMod`).

Keep one of these in the team notebook. The rest of this lecture assumes an NTT-friendly prime.

4. Formal power series and Newton's method

A formal power series is `A(x) = Σ a_i x^i` with no convergence questions. We only ever store $A \bmod x^n$, the first n coefficients. The basic facts:

- `A` is invertible if and only if $a_0 \neq 0$.

- $\ln A$ is defined when $a_0 = 1$, as $\ln A = \int A'/A$, and then $(\ln A)_0 = 0$.
- $\exp A$ is defined when $a_0 = 0$, and then $(\exp A)_0 = 1$.
- Derivative and integral are coefficient-wise. The integral divides by $1..n$, so it needs $n < p$.

Newton's iteration. Suppose we want B with $F(B) \equiv 0 \pmod{x^n}$ for some operator F, and we know B_0 with $F(B_0) \equiv 0 \pmod{x^m}$. Expanding $F(B)$ around B_0 (Taylor series with formal derivatives), the error term is $O((B - B_0)^2)$, and since $B - B_0 \equiv 0 \pmod{x^m}$ that error is $0 \pmod{x^{2m}}$. Hence

$$B_1 = B_0 - F(B_0) / F'(B_0) \quad \text{satisfies} \quad F(B_1) \equiv 0 \pmod{x^{2m}}.$$

Every step **doubles the number of correct coefficients**. If one step at precision m costs O(M(m)), where M(m) = O(m log m) is the cost of a multiplication, the total cost is $M(n) + M(n/2) + M(n/4) + \ldots = O(M(n)) = O(n \log n)$.

4.1 Inverse

Take $F(B) = 1/B - A$. Then $F'(B) = -1/B^2$ and

$$B \leftarrow B - (1/B - A)/(-1/B^2) = B + B - A\,B^2 = B\,(2 - A\,B) \pmod{x^{2m}}.$$

Start with $B = a_0^{-1}$. Each step needs only the first $2m$ coefficients of A, and does two multiplications. A useful sanity check is that $A \cdot B \equiv 1 \pmod{x^{2m}}$ after each step.

4.2 Logarithm

$\ln A = \int (A' \cdot A^{-1})$. Compute the inverse to precision n, multiply by the derivative, truncate to n − 1 terms and integrate. The cost is one inverse plus one multiplication.

If $a_0 \neq 1$ but $a_0 \neq 0$, write $A = a_0 \cdot (A/a_0)$. Over F_p, $\ln a_0$ is not defined for a general constant, but for the uses below (computing A^k) you can handle the constant separately: $A^k = a_0^k \cdot (A/a_0)^k$.

4.3 Exponential

Solve $\ln B - A = 0$, so $F'(B) = 1/B$:

$$B \leftarrow B - (\ln B - A) \cdot B = B\,(1 - \ln B + A) \pmod{x^{2m}}.$$

Each step computes a log at precision 2m (which contains an inverse) and one multiplication, so exp is O(n log n) with a large constant: about 0.4 s for $n = 2^{18}$ with the reference code. If the limits are tight, there are faster methods (exp via online/"relaxed" convolution with CDQ divide

and conquer from Lecture 5, or a combined Newton iteration that maintains the inverse alongside), but the simple version passes most problems.

4.4 Powers and roots

- `A^k` for huge k: if $a_0 = 1$, `A^k = exp(k · ln A)`, with k reduced modulo p (it multiplies coefficients). In general, factor out the lowest term $c \cdot x^t$: `A = c x^t (1 + …)`, so `A^k = c^k x^{tk} (1 + …)^k`. Here `c^k` uses k mod (p − 1), the shift uses the real k, and the series part uses k mod p.
- `√A` with $a_0 = 1$: Newton on $B^2 - A$, giving `B ← (B + A/B)/2`. Codeforces 438E needs exactly this.

5. Why log and exp count structures

Use exponential generating functions (EGFs) for labelled objects: `A(x) = Σ a_k x^k / k!`, where `a_k` counts structures on the label set `{1..k}`.

Exponential formula. If every structure decomposes uniquely into a *set* of connected components, then

```
All(x) = exp(Connected(x)),   equivalently   Connected(x) = ln All(x).
```

The intuition: the product of EGFs counts ways to split the labels between two parts, and `C^j / j!` counts *unordered* collections of j components.

Worked example. Graphs on k labelled vertices: `g_k = 2^{k(k–1)/2}`. Then `c_k = k! · [x^k] ln(Σ g_k x^k/k!)` gives 1, 1, 4, 38, 728, 26704, …, the number of connected labelled graphs. The exponent `k(k–1)/2` is up to $5 \cdot 10^9$ for $k = 10^5$, so reduce it modulo `p – 1` (Fermat), never modulo `p`.

Other standard translations:

Combinatorial operation	EGF / OGF operation
set of connected objects (labelled)	exp
sequence of objects	1/(1 − A) (inverse)
cycle of objects (labelled)	ln(1/(1 − A))
multiset of unlabelled objects	Euler transform exp(Σ_j A(x^j)/j)
k-fold product	A^k = exp(k ln A)

Lecture 7 (advanced combinatorics) builds on this table.

6. Linear recurrences and Berlekamp–Massey

6.1 The k-th term (Kitamasa)

Let $s_i = \Sigma_{j=1}^{L} c_j s_{i-j}$, with characteristic polynomial $Q(x) = x^L - c_1 x^{L-1} - \dots - c_L$. The shift operator satisfies Q, so $s_k = \Sigma_j r_j s_j$, where $r(x) = x^k \bmod Q(x)$. Computing r by square-and-multiply with polynomial multiplication modulo Q costs $O(L^2 \log k)$ with schoolbook multiplication, which is fine for L up to about 2000. With NTT multiplication and NTT-based polynomial division it costs $O(L \log L \log k)$. Either is much better than the $O(L^3 \log k)$ of matrix power.

6.2 Berlekamp–Massey

Problem. Given terms $s_0..s_{n-1}$ over a field, find the shortest recurrence (the smallest L) that generates all of them.

Theorem (uniqueness). If the sequence satisfies some recurrence of order L and $n \geq 2L$, the shortest recurrence found from those n terms is the true one.

Algorithm. Process terms one at a time while maintaining the current recurrence C (as the polynomial $1 + C_1 x + \dots + C_L x^L$, so that $\Sigma_{j=0}^{L} C_j s_{i-j} = 0$). We also keep the previous version B that was replaced, together with its discrepancy b and the number of steps m since it was replaced.

- Compute the discrepancy $d = \Sigma_{j=0}^{L} C_j s_{i-j}$. If $d = 0$, C still works.
- Otherwise set $C \leftarrow C - (d/b) \cdot x^m \cdot B$. By construction, B's own discrepancy was b at position $i - m$. Shifting B by m and scaling by d/b cancels exactly the new discrepancy without disturbing earlier positions.
- If $2L \leq i$, the length must grow: $L \leftarrow i + 1 - L$, and the old C becomes the new B.

The algorithm runs in $O(n^2)$. It needs division, so it requires a **field** (a prime modulus). Over $\mathbb{Z}/2^{32}$ or other composite moduli the result is meaningless.

6.3 The contest workflow

1. You suspect the answer satisfies a fixed-order linear recurrence. Typical sources are DP whose transition is a fixed linear map on a bounded state (tilings of a k × n strip, walks of length n in a fixed graph, digit DP over n with a fixed automaton).
2. Write the slow DP and compute the first 2L + 20 terms. If you do not know L, generate as many as the brute force allows (500–2000).
3. Run BM. **Verify** that the recurrence predicts the extra terms. If it does not, you did not have enough terms or the sequence is not linearly recurrent.
4. Use Kitamasa to get term n.

The order L is bounded by the dimension of the DP state space, which tells you how many terms you need.

7. Lagrange interpolation

A polynomial of degree ≤ d is determined by d + 1 values:

```
P(x) = Σ_i y_i Π_{j≠i} (x − x_j)/(x_i − x_j).
```

For arbitrary points, evaluating at one x costs $O(d^2)$ (or $O(d \log^2 d)$ with fast multipoint techniques). The **consecutive case** `x_i = i, i = 0..d` is the one that matters in contests. There, the denominator is `Π_{j≠i}(i − j) = i! · (d − i)! · (−1)^{d−i}`, and the numerators are prefix and suffix products of `(x − j)`. One evaluation costs O(d).

Why answers are polynomials.

- `Σ_{i=1}^{n} i^k` is a polynomial in n of degree k + 1 (Faulhaber). This is Codeforces 622F, with $n \le 10^9$ and $k \le 10^6$.
- If `dp[v]` is a polynomial in v of degree t, then its prefix sum is a polynomial of degree t + 1. So DPs of the form "choose values up to a huge bound X in a monotone way" are polynomial in X of degree equal to the number of summation layers. Interpolate from small X.
- Counting colourings of a fixed structure with n colours gives a polynomial in n (the chromatic polynomial).

Check before trusting. Compute d + 2 values and verify that the (d+1)-th finite difference is zero, or simply check one extra sample against the interpolated value.

8. Reference implementations

8.1 Polynomial library with a demo: connected labelled graphs

Input: n. Output: for k = 1..n, the number of connected labelled graphs on k vertices modulo 998244353. All operations were tested against naive $O(n^2)$ versions on random inputs, including `exp(log A) = A` and the identity `(exp A)′ = A′ · exp A`.

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
using poly = vector<int>;

const int MOD = 998244353, ROOT = 3;  // MOD = 119 * 2^23 + 1

ll power(ll b, ll e) {
```

```
    ll r = 1; b %= MOD; if (b < 0) b += MOD;
    for (; e; e >>= 1, b = b * b % MOD) if (e & 1) r = r * b % MOD;
    return r;
}

// In-place NTT; n must be a power of two dividing 2^23.
void ntt(poly& a, bool invert) {
    int n = a.size();
    for (int i = 1, j = 0; i < n; i++) {          // bit-reversal permutation
        int bit = n >> 1;
        for (; j & bit; bit >>= 1) j ^= bit;
        j ^= bit;
        if (i < j) swap(a[i], a[j]);
    }
    static vector<int> rt;                         // rt[k + j] = w_{2k}^j
    if ((int)rt.size() < n) {
        rt.assign(max(n, 2), 1);
        for (int k = 1; k < n; k <<= 1) {
            ll w = power(ROOT, (MOD - 1) / (2 * k));
            for (int j = 0; j < k; j++)
                rt[k + j] = j ? (ll)rt[k + j - 1] * w % MOD : 1;
        }
    }
    for (int k = 1; k < n; k <<= 1)
        for (int i = 0; i < n; i += 2 * k)
            for (int j = 0; j < k; j++) {
                int u = a[i + j], v = (ll)a[i + j + k] * rt[k + j] % MOD;
                a[i + j] = u + v >= MOD ? u + v - MOD : u + v;
                a[i + j + k] = u - v < 0 ? u - v + MOD : u - v;
            }
    if (invert) {
        reverse(a.begin() + 1, a.end());           // inverse = forward + reverse
        ll inv = power(n, MOD - 2);
        for (int& x : a) x = x * inv % MOD;
    }
}

poly multiply(poly a, poly b) {
    if (a.empty() || b.empty()) return {};
    int need = a.size() + b.size() - 1, n = 1;
    while (n < need) n <<= 1;
    if (min(a.size(), b.size()) <= 32) {           // schoolbook for tiny inputs
        poly c(need);
        for (size_t i = 0; i < a.size(); i++)
            for (size_t j = 0; j < b.size(); j++)
                c[i + j] = (c[i + j] + (ll)a[i] * b[j]) % MOD;
        return c;
    }
    a.resize(n); b.resize(n);
    ntt(a, false); ntt(b, false);
    for (int i = 0; i < n; i++) a[i] = (ll)a[i] * b[i] % MOD;
    ntt(a, true);
    a.resize(need);
    return a;
}
```

```
// 1 / A mod x^n.  Requires A[0] != 0.  Newton: B <- B (2 - A B).
poly inverse(const poly& A, int n) {
    poly B{(int)power(A[0], MOD - 2)};
    for (int m = 1; m < n; m <<= 1) {
        poly a(A.begin(), A.begin() + min((int)A.size(), 2 * m));
        poly ab = multiply(a, B);
        ab.resize(2 * m);
        for (int& x : ab) x = x ? MOD - x : 0;     // -A B
        ab[0] = (ab[0] + 2) % MOD;                 // 2 - A B
        B = multiply(B, ab);
        B.resize(2 * m);
    }
    B.resize(n);
    return B;
}

poly derivative(const poly& A) {
    poly d(max((int)A.size() - 1, 0));
    for (size_t i = 1; i < A.size(); i++) d[i - 1] = (ll)A[i] * i % MOD;
    return d;
}

poly integral(const poly& A) {
    int n = A.size();
    poly r(n + 1), inv(n + 2, 1);
    for (int i = 2; i <= n; i++) inv[i] = (ll)(MOD - MOD / i) * inv[MOD % i] % MOD;
    for (int i = 0; i < n; i++) r[i + 1] = (ll)A[i] * inv[i + 1] % MOD;
    return r;
}

// ln A mod x^n.  Requires A[0] == 1.
poly logarithm(const poly& A, int n) {
    poly a(A.begin(), A.begin() + min((int)A.size(), n));
    poly q = multiply(derivative(a), inverse(a, n));
    q.resize(n - 1 > 0 ? n - 1 : 0);
    poly r = integral(q);
    r.resize(n);
    return r;
}

// exp A mod x^n.  Requires A[0] == 0.  Newton: B <- B (1 - ln B + A).
poly exponent(const poly& A, int n) {
    poly B{1};
    for (int m = 1; m < n; m <<= 1) {
        poly l = logarithm(B, 2 * m);
        for (int i = 0; i < 2 * m; i++) {
            int ai = i < (int)A.size() ? A[i] : 0;
            l[i] = ((ll)ai - l[i] + MOD) % MOD;
        }
        l[0] = (l[0] + 1) % MOD;
        B = multiply(B, l);
        B.resize(2 * m);
    }
    B.resize(n);
```

```
    return B;
}

int main() {
    int n;
    cin >> n;
    vector<ll> fact(n + 1, 1), ifact(n + 1, 1);
    for (int i = 1; i <= n; i++) fact[i] = fact[i - 1] * i % MOD;
    ifact[n] = power(fact[n], MOD - 2);
    for (int i = n; i > 0; i--) ifact[i - 1] = ifact[i] * i % MOD;
    poly G(n + 1);
    for (int k = 0; k <= n; k++)                    // EGF coefficient 2^{C(k,2)} / k!
        G[k] = power(2, (ll)k * (k - 1) / 2 % (MOD - 1)) * ifact[k] % MOD;
    poly C = logarithm(G, n + 1);                   // connected = ln(all)
    for (int k = 1; k <= n; k++) cout << C[k] * fact[k] % MOD << "\n";
}
```

8.2 Berlekamp–Massey and Kitamasa

Input: `n k` followed by `s_0 .. s_{n−1}`. Output: the recurrence order and coefficients, then `s_k`. This was tested on random recurrences of order ≤ 8 (given 2L terms, BM reproduced all later terms) and on Fibonacci at k = 10^{18} against fast doubling.

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
const ll MOD = 998244353;

ll power(ll b, ll e) {
    ll r = 1; b %= MOD; if (b < 0) b += MOD;
    for (; e; e >>= 1, b = b * b % MOD) if (e & 1) r = r * b % MOD;
    return r;
}

// Returns c[0..L-1] with s[i] = sum_j c[j] * s[i-1-j] for all i >= L.
vector<ll> berlekampMassey(const vector<ll>& s) {
    int n = s.size(), L = 0, m = 0;
    vector<ll> C(n + 1), B(n + 1), T;
    C[0] = B[0] = 1;
    ll b = 1;                                       // discrepancy when B was saved
    for (int i = 0; i < n; i++) {
        m++;
        ll d = s[i] % MOD;                          // discrepancy of C at position i
        for (int j = 1; j <= L; j++) d = (d + C[j] * s[i - j]) % MOD;
        if (d == 0) continue;
        T = C;
        ll coef = d * power(b, MOD - 2) % MOD;
        for (int j = m; j <= n; j++) C[j] = (C[j] - coef * B[j - m] % MOD + MOD) % MOD;
        if (2 * L > i) continue;
        L = i + 1 - L; B = T; b = d; m = 0;
    }
    vector<ll> res(L);
```

```
    for (int j = 0; j < L; j++) res[j] = (MOD - C[j + 1]) % MOD;
    return res;
}

// k-th term (0-indexed) of s, given recurrence c. Computes x^k mod charpoly: O(L^2 log
k).
ll linearRecurrenceKth(const vector<ll>& s, const vector<ll>& c, ll k) {
    int L = c.size();
    if (L == 0) return 0;
    auto mulmod = [&](const vector<ll>& a, const vector<ll>& b) {
        vector<ll> r(2 * L - 1);
        for (int i = 0; i < L; i++) for (int j = 0; j < L; j++)
            r[i + j] = (r[i + j] + a[i] * b[j]) % MOD;
        for (int i = 2 * L - 2; i >= L; i--)      // x^L = sum c[j] x^{L-1-j}
            for (int j = 0; j < L; j++)
                r[i - 1 - j] = (r[i - 1 - j] + r[i] * c[j]) % MOD;
        r.resize(L);
        return r;
    };
    vector<ll> res(L), base(L);
    res[0] = 1;                                   // x^0
    if (L == 1) base[0] = c[0]; else base[1] = 1; // x mod charpoly
    for (; k; k >>= 1, base = mulmod(base, base)) if (k & 1) res = mulmod(res, base);
    ll ans = 0;
    for (int i = 0; i < L; i++) ans = (ans + res[i] * s[i]) % MOD;
    return ans;
}

int main() {
    int n; ll k;
    cin >> n >> k;
    vector<ll> s(n);
    for (auto& x : s) cin >> x;
    vector<ll> c = berlekampMassey(s);
    cout << c.size() << "\n";
    for (ll x : c) cout << x << " ";
    cout << "\n" << linearRecurrenceKth(s, c, k) << "\n";
}
```

8.3 Lagrange interpolation: Codeforces 622F

Input: `n k`. Output: `Σ_{i=1}^{n} i^k mod 10⁹+7`. This was verified against direct summation for n ≤ 300, k ≤ 8, and on the official samples.

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
const ll MOD = 1'000'000'007;

ll power(ll b, ll e) {
    ll r = 1; b %= MOD; if (b < 0) b += MOD;
    for (; e; e >>= 1, b = b * b % MOD) if (e & 1) r = r * b % MOD;
```

```
    return r;
}

// y[i] = P(i) for i = 0..d; returns P(x) mod MOD in O(d) (plus one inverse).
ll lagrangeConsecutive(const vector<ll>& y, ll x) {
    int d = (int)y.size() - 1;
    if (x <= d) return y[x];
    vector<ll> pre(d + 2, 1), suf(d + 2, 1), ifact(d + 1, 1);
    for (int i = 0; i <= d; i++) pre[i + 1] = pre[i] * ((x - i) % MOD) % MOD;
    for (int i = d; i >= 0; i--) suf[i] = suf[i + 1] * ((x - i) % MOD) % MOD;
    ll f = 1;
    for (int i = 1; i <= d; i++) f = f * i % MOD;
    ifact[d] = power(f, MOD - 2);
    for (int i = d; i > 0; i--) ifact[i - 1] = ifact[i] * i % MOD;
    ll ans = 0;
    for (int i = 0; i <= d; i++) {
        ll term = y[i] * pre[i] % MOD * suf[i + 1] % MOD * ifact[i] % MOD * ifact[d - i]
% MOD;
        ans = ((d - i) & 1) ? (ans - term + MOD) % MOD : (ans + term) % MOD;
    }
    return ans;
}

int main() {                        // Codeforces 622F: sum_{i=1}^{n} i^k mod 1e9+7
    ll n; int k;
    cin >> n >> k;
    vector<ll> y(k + 2, 0);         // S(x) = sum_{i<=x} i^k has degree k+1
    for (int i = 1; i <= k + 1; i++) y[i] = (y[i - 1] + power(i, k)) % MOD;
    cout << lagrangeConsecutive(y, n) << "\n";
}
```

For $k = 10^6$, the `power` calls cost O(k log k). A linear sieve that computes `i^k` only for primes (and multiplicatively for composites) makes this O(k), but that is rarely needed.

9. Common mistakes

- **Wrong preconditions.** Inverse needs $a_0 \neq 0$, log needs $a_0 = 1$, exp needs $a_0 = 0$. Normalise first, and handle a leading `x^t` factor separately for powers.
- **Precision leaks in Newton loops.** After each step, truncate to exactly `2m` coefficients. Keeping garbage high-order terms is harmless for correctness only if you truncate at the end, but it doubles the running time. Truncating too early gives wrong answers.
- **Reducing exponents modulo p.** In `2^{k(k-1)/2}` the exponent is reduced modulo `p - 1`. In `exp(k ln A)` the multiplier `k` is reduced modulo `p`. Mixing these up is a classic WA.
- **NTT length overflow.** 998244353 supports transform lengths up to $2^{23} \approx 8.4 \cdot 10^6$. A product of two length-$5 \cdot 10^6$ polynomials does not fit.
- **Using BM over a composite modulus.** BM divides by discrepancies, which is only valid in a field.
- **Too few terms for BM.** With fewer than 2L terms, BM happily returns a shorter, wrong recurrence. Always verify the recurrence on extra terms.

- **Interpolating something that is not a polynomial.** Piecewise polynomials (for example, a DP over values with breakpoints at the input numbers) must be interpolated piece by piece.
- **Recomputing the root table in every NTT call.** It is a silent 2× slowdown. Cache it.

10. Practice problems

1. Library Checker — *Inv of Formal Power Series*: https://judge.yosupo.jp/problem/inv_of_formal_power_series
2. Library Checker — *Log of Formal Power Series*: https://judge.yosupo.jp/problem/log_of_formal_power_series
3. Library Checker — *Exp of Formal Power Series*: https://judge.yosupo.jp/problem/exp_of_formal_power_series
4. Library Checker — *Pow of Formal Power Series*: https://judge.yosupo.jp/problem/pow_of_formal_power_series
5. Library Checker — *Find Linear Recurrence*: https://judge.yosupo.jp/problem/find_linear_recurrence
6. Library Checker — *Kth term of Linearly Recurrent Sequence*: https://judge.yosupo.jp/problem/kth_term_of_linearly_recurrent_sequence
7. Codeforces 622F — *The Sum of the k-th Powers* (Lagrange): https://codeforces.com/problemset/problem/622/F
8. Codeforces 438E — *The Child and Binary Tree* (inverse and square root of a series): https://codeforces.com/problemset/problem/438/E
9. Codeforces 1096G — *Lucky Tickets* (power of a polynomial via NTT): https://codeforces.com/problemset/problem/1096/G
10. Codeforces 528D — *Fuzzy Search* (FFT string matching): https://codeforces.com/problemset/problem/528/D
11. Codeforces 1251F — *Red-White Fence* (products of polynomials): https://codeforces.com/problemset/problem/1251/F
12. ICPC WF 2015 — *Tile Cutting* (FFT convolution of divisor counts), on the Kattis archive.

11. Further reading

- cp-algorithms, "Operations on polynomials and series": https://cp-algorithms.com/algebra/polynomial.html
- cp-algorithms, "Berlekamp–Massey" and "Lagrange interpolation" related articles: https://cp-algorithms.com/
- KACTL's `PolyInverse`, `BerlekampMassey`, `LinearRecurrence` and `FastFourierTransformMod`: https://github.com/kth-competitive-programming/kactl
- P. Flajolet and R. Sedgewick, *Analytic Combinatorics* (free online), Part A, for the symbolic method behind the exp/log table.

- J. L. Massey, "Shift-register synthesis and BCH decoding", IEEE Trans. Inf. Theory, 1969 — the original BM paper.